

AUTOMATED PODCAST TRANSCRIPTION AND TOPIC SEGMENTATION

OBJECTIVES

The primary objective of this project is to convert podcast audio into structured and meaningful knowledge using Artificial Intelligence techniques.

- **Accurate Transcription:** To develop an automated system for transcribing podcast audio into precise text using advanced speech recognition models.
- **Audio Optimization:** To implement preprocessing and noise reduction techniques that enhance the quality of the input audio before transcription.
- **Semantic Segmentation:** To apply Natural Language Processing (NLP) to divide the transcript into coherent, topic-based sections.
- **Automated Summarization:** To generate concise summaries and extract key discussion points for each segment to save user time.
- **User-Centric Interface:** To design a web-based dashboard for uploading audio, playing podcasts, and visualizing segmented results with timestamps.

APPROACH

The project follows a modular, step-by-step pipeline to ensure high accuracy and efficiency:

1. Audio Preprocessing Phase

- **Format Normalization:** Raw audio files (MP3, AAC, M4A) are converted into the standard .WAV format for processing.
- **Signal Cleaning:** Background static and noise are filtered out to improve the word recognition rate.
- **Resampling:** Audio is downsampled to exactly 16kHz mono, which is the optimal frequency for the speech-to-text engine.

2. Transcription & NLP Phase

- **Speech-to-Text (ASR):** The cleaned audio is processed using the **OpenAI Whisper** model, which generates a transcript with millisecond-accurate timestamps.
- **Topic Segmentation:** The transcript is analyzed for semantic boundaries where the speaker shifts topics.
- **Summarization & Keywords:** A dedicated NLP module (using BART and NLTK) generates abstractive summaries and extracts relevant keywords for each identified topic.

3. System Architecture

- The architecture consists of a **React frontend** for user interaction, a **Flask REST API** for orchestration, and a background pipeline for heavy AI computation.

SYSTEM SETUP & REQUIREMENTS

To set up the project locally, ensure the following environment is configured:

1. Software Prerequisites

- **Python 3.8+:** For the backend API and AI models.
- **Node.js:** For running the React-based frontend.
- **FFmpeg:** Essential system-level tool for audio format conversion and resampling.

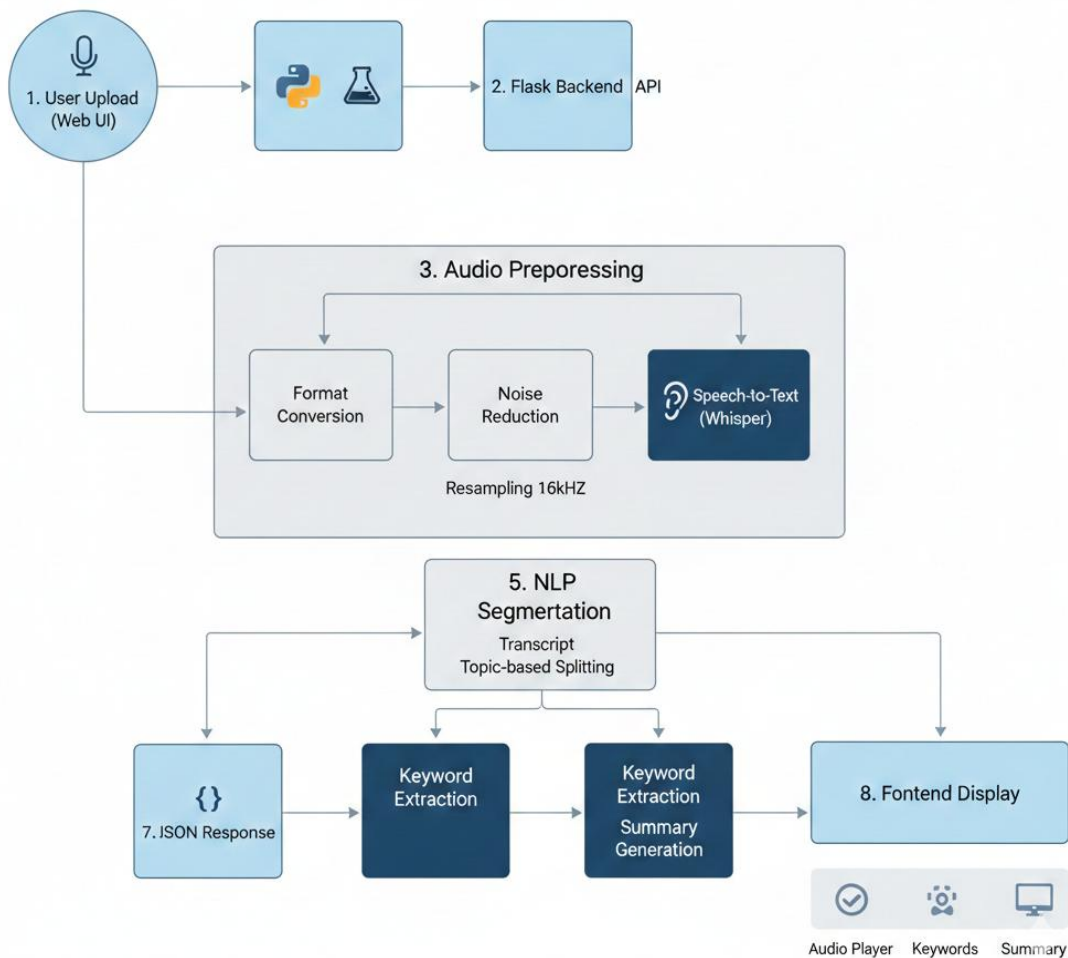
3. Library Dependencies

- **Backend:** Flask, OpenAI-Whisper, Librosa, NLTK, Transformers (for BART model).
- **Frontend:** React.js, Tailwind CSS, Axios.

USAGE / HOW TO RUN

- **Initialize Backend:** Navigate to the backend/ directory and run `python app.py`. This starts the Flask server and loads the AI models.
- **Initialize Frontend:** Navigate to the frontend/ directory and run `npm start` to launch the web dashboard
- **Process Audio:** Upload a podcast file (MP3, WAV, or M4A) through the **UploadCard** interface.
- **Analyze Results:** Once the pipeline completes, the **PodcastResult** page will display the audio player alongside a timeline of segments, summaries, and keywords.

FLOW DIAGRAM



TROUBLESHOOTING

- **File Upload Failure:** Ensure the file does not exceed size limits and is in a supported audio format.
- **Slow Processing:** Transcription of long files (1 hour+) is compute-heavy. If using a CPU, expect higher latency; using a GPU is highly recommended.

- **Transcription Errors:** High background noise or very thick regional accents may occasionally result in minor word errors.
 - **Model Loading Issues:** Ensure you have a stable internet connection during the first run to allow the system to download the pre-trained Whisper and BART weights.
-

CHALLENGES AND LIMITATIONS

- **Hardware Constraints:** The Whisper model requires significant RAM and GPU power for real-time performance on long audio files.
 - **Connectivity:** Currently depends on internet access for certain NLP libraries or API-based modules.
 - **Latency:** There is a notable delay between audio upload and result generation due to the sequential nature of transcription and summarization.
 - **Acoustic Variability:** Heavy noise or overlapping speakers can sometimes blur the accuracy of topic boundaries.
-

FUTURE SCOPE

- **Speaker Diarization:** Implementing logic to identify and label different speakers (e.g., Speaker A vs. Speaker B).
- **Real-time Processing:** Adapting the pipeline to transcribe live audio streams as they occur.

- **Cloud Integration:** Deploying to AWS or Azure to utilize auto-scaling and high-performance GPU instances.
- **Advanced Search:** Enabling users to search for specific phrases and jump directly to that timestamp in the audio player.

REFERENCES

- **Dataset Source:** The American Life Podcast Dataset and Kaggle Podcast Transcripts
- **AI Models:** OpenAI Whisper for ASR and BART for Abstractive Summarization.
- **Frameworks:** React (Frontend), Flask (Backend), Tailwind CSS (Styling)
- **Mentorship:** Delivered through the Infosys Springboard Virtual Internship Program.